



Lab Sheet 8

Digital Image Processing Using MATLAB

↗ Prerequisite

Before starting this section, students should know..

1. A fundamental knowledge on MATLAB

↗ Outcomes

After finishing this lab students should be able to.....

1. understand basic image processing functions available in MATLAB.
2. understand Conversion of images from one format to another
3. filtering used in image processing
4. understand Image Histogram and Histogram Equalization

➡ Outlines of this Lab

1. Theoretical Background
2. Lab Session 8.1: Basic image processing functions and their use:
3. Lab Session 8.2: **Conversion of images** from one format to another:
4. Lab Session 8.3: Filtering
5. Lab Session 8.4: Image Histogram and Histogram Equalization
6. Home work

Theoretical Background

Image

A digital image is a rectangular array of pixels (picture elements). Each pixel represents the color (or brightness for black and white photos) at a single point in the image. So a pixel is like a tiny dot of a particular color.

Image Processing

Image processing involves changing the nature of an image in order to either

- (a) improve its pictorial information for human interpretation,
- (b) render it more suitable for autonomous machine perception.

We shall be concerned with digital image processing, which involves using a computer to change the nature of a digital image. It is necessary to realize that these two aspects represent two separate but equally important aspects of image processing.

Five Basic Types of Images

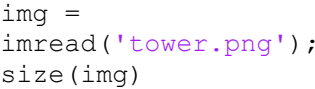
Grayscale	A grayscale image m pixels tall and n pixels wide is represented as a matrix of double datatype of size $m \times n$. Element values (such as $img(m,n)$) denote the pixel grayscale intensities in $[0,1]$ with 0=black and 1=white. Grayscale images are also known as black-and-white images, are composed exclusively of shades of gray, varying from black at the weakest intensity to white at the strongest.
Truecolor RGB	A truecolor RGB image is represented as a three-dimensional $m \times n \times 3$ double matrix. Each pixel has red, green, blue components along the third dimension with values in $[0,1]$, for example, the color components of pixel (m,n) are $img(m,n,1)$ = red, $img(m,n,2)$ = green, $img(m,n,3)$ = blue. <i>Color image = Red image + Green image + Blue image</i>
Indexed	Indexed images are represented with an index matrix of size $m \times n$ and a colormap matrix of size $k \times 3$. The colormap holds all colors used in the image and the index matrix represents the pixels by referring to colors in the colormap. For example, if the 19th color is magenta $cmap(19,:)=[1,0,1]$, then $img(m,n)=19$ is a magenta-colored pixel.
Binary	A binary image is represented by an $m \times n$ logical matrix where pixel values are 1 (true) or 0 (false).
uint8	This type uses less memory and some operations compute faster than with double types.

Note that the value range depends on the data type class: for example, the $[0,1]$ is for double data type. If it's uint8, the range that Matlab interprets is $[0,255]$, and so on.

Lab Session 8.1: Basic image processing functions and their use:

To illustrate the use of basic functions, we place a test image named **tower.png** in the current folder of MATLAB and apply the functions sequentially.

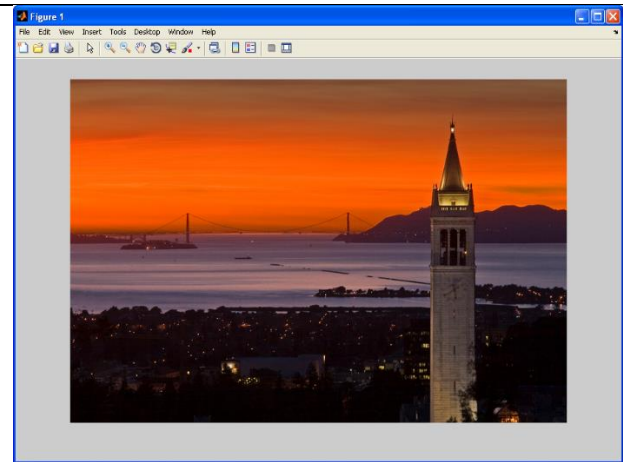
Name	MATLAB Code	Output
imfinfo()	<code>imfinfo('tower.png')</code>	Filename: 'C:\Users\bkm.AD\Documents\MATLAB\tower.png' FileModDate: '28-Nov-2015 17:50:29' FileSize: 468861

<i>Gives information about the image.</i>		Format: 'png' FormatVersion: [] Width: 748 Height: 499 BitDepth: 24 ColorType: 'truecolor' FormatSignature: [137 80 78 71 13 10 26 10] Colormap: [] Histogram: [] InterlaceType: 'none' Transparency: 'none' SimpleTransparencyData: [] BackgroundColor: [] RenderingIntent: [] Chromaticities: [] Gamma: [] XResolution: 3780 YResolution: 3780 ResolutionUnit: 'meter' XOffset: [] YOffset: [] OffsetUnit: [] SignificantBits: [] ImageModTime: '13 Apr 2014 18:07:41 +0000' Title: " Author: " Description: " Copyright: " CreationTime: [] Software: " Disclaimer: " Warning: " Source: Comment: " OtherText: {'Creation time' " }
 <i>Read image from graphics file</i>	 <pre>img = imread('tower.png'); size(img)</pre>	<pre>ans = 499 748 3</pre> It's a 499x748 matrix with 3 RGB channels. Note: An RGB image has three channels: red, green, and blue. A part of the matrix looks like this: <pre>24 40 73 108 129 108 96 100 109 114 108 109 62 29 56 97 107 110 104 103 105 106 110 110 111 105 ... 3 2 2 1 0 0 1 1 0 1 2 4 2 1 0 1 3 2 0 0 0 1 1 2 1 0 ...</pre>

imshow()

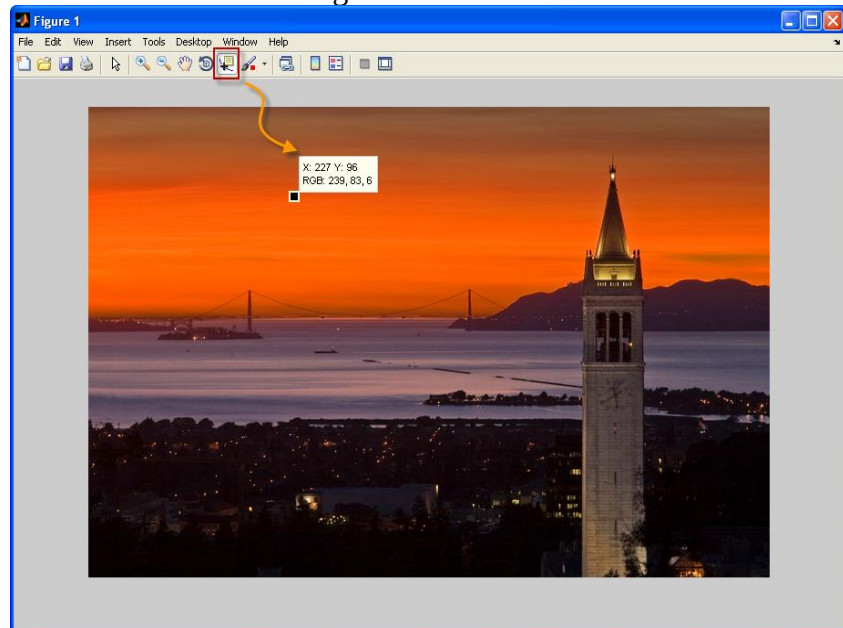
Shows an image in standard 8-bit format.

```
imshow('tower.png')
```



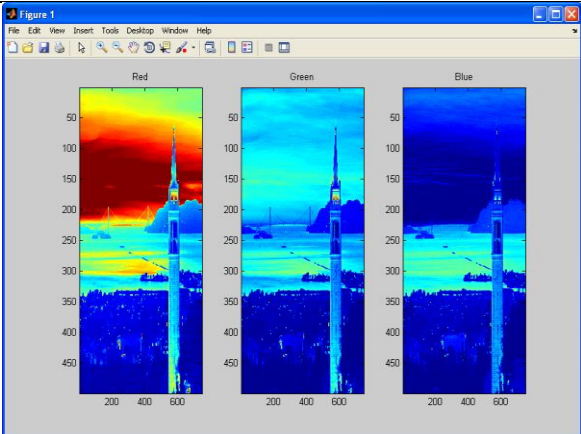
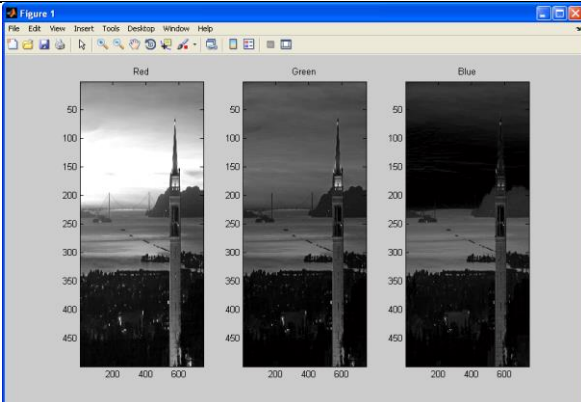
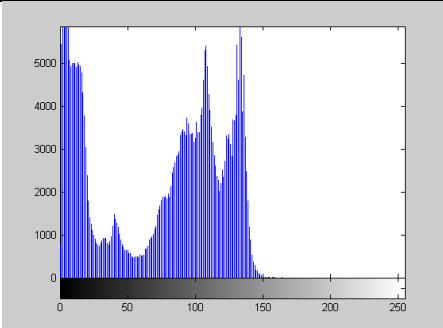
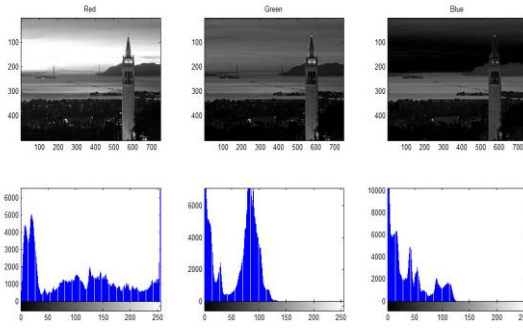
We can check the RGB values with (x,y) coordinates of a pixel:

1. Select "Data Cursor" icon from the top menu
2. Click the mouse on the image



Notice each pixel is a 3-dimensional vector with values in the range [0,255]. The 3 numbers displayed is the amount of RGB.

Actually, a color image is a combined image of 3 grayscale images. So, we can display the individual RGB components of the image using the following script:

	<pre>subplot(131); imagesc(img(:,:,1)); title('Red'); subplot(132); imagesc(img(:,:,2)); title('Green'); subplot(133); imagesc(img(:,:,3)); title('Blue');</pre>	
colormap gray <i>The picture turns into a grayscale:</i>	We must use gray colormap to display 3 images properly, otherwise each value in each pixel will follow some default color. <pre>>> colormap gray</pre>	
imwrite()	<code>imwrite(tower.png, 't.png', 'png');</code>	The image 'tower.png' is saved as 't.png'.
imhist() <i>Display a histogram of image data.</i>	<pre>img = imread('tower.png'); gray = rgb2gray(img); imhist(gray);</pre> OR <pre>subplot(231); imagesc(img(:,:,1)); title('Red'); subplot(232); imagesc(img(:,:,2)); title('Green'); subplot(233); imagesc(img(:,:,3)); title('Blue');</pre> <pre>colormap gray</pre> <pre>subplot(234); imhist(img(:,:,1)); subplot(235); imhist(img(:,:,2)); subplot(236); imhist(img(:,:,3));</pre>	 

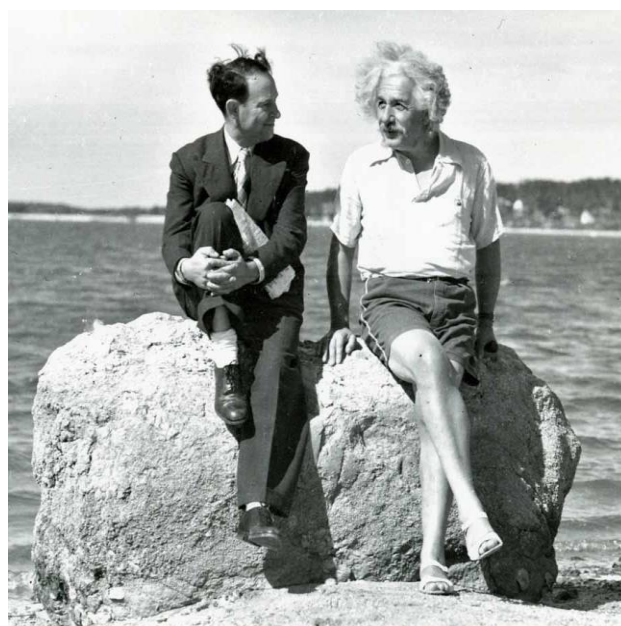
imadjust() <i>Adjust image intensity values</i>	<pre>img = imread('tower.png'); gray = rgb2gray(img); adj_img = imadjust(gray, [0.3,0.7],[]); subplot(121); imshow(gray); title('input'); subplot(122); imshow(adj_img); title('adjusted');</pre>	input  adjusted  $J = \text{imadjust}(I, [\text{LOW_IN}; \text{HIGH_IN}], [\text{LOW_OUT}; \text{HIGH_OUT}])$ values below LOW_IN map to LOW_OUT, and those above HIGH_IN map to HIGH_OUT. You can use an empty matrix ([]) for [LOW_IN; HIGH_IN] or for [LOW_OUT; HIGH_OUT] to specify the default of [0 1].
	<pre>img = imread('tower.png'); blueImg=img(:,:,3); subplot(121);imshow(blueI mg); colormap gray title('Original Blue Component'); adj_img = imadjust(blueImg, [0,0.5],[0 1]); subplot(122);imshow(adj_i mg); colormap gray title('Adjusted Blue Component');</pre>	Original Blue Component  Adjusted Blue Component 

Lab Session 8.2: Conversion of images from one format to another:

8.2.1 RGB image to Grayscale Image

Place the image 'Einstein.jpg' in the current folder.

MATLAB Code	Output
<pre>img = imread('Einstein.png'); imshow(img);</pre>	Output is the following figure.



First check whether it is a grayscale image or not, using **size()**:

MATLAB Code	Output
>> size(img)	ans = 845 915 3

So, the image is not grayscale. So, convert it to real-grayscale using **rgb2gray()** which converts the truecolor image RGB to the grayscale intensity image. The **rgb2gray()** converts RGB images to grayscale by eliminating the hue and saturation information while retaining the luminance.

MATLAB Code	Output
img = imread('Einstein.png'); gray = rgb2gray(img); imshow(gray); size(gray)	ans = 845 915 Now, this is real grayscale image!

Investigation:

Try following code:

```
img = imread('Einstein.png');  
img(:,:,1)=0; imshow(img);
```

Can you explain the change in the output?

To understand the change, try to observe RGB components separately:

```
subplot(131); imagesc(img(:,:,1));  
subplot(132); imagesc(img(:,:,2));  
subplot(133); imagesc(img(:,:,3));  
colormap gray
```

Can you explain now? If you do, try the following code as well:

```
img = imread('Einstein.png');  
img(:,:,2)=0; img(:,:,3)=0; imshow(img);
```

8.2.2 RGB to Binary image:

MATLAB Code	Output
img = imread('tower.png'); bw_img = im2bw(img); subplot(121); imshow(img); title('input RGB image'); subplot(122); imshow(bw_img); title('binary image');	input RGB image  binary image 

--	--

8.2.3 RGB to indexed image:

MATLAB Code	Output
<pre>[IND,map] = rgb2ind(img,128); figure('Name','Indexed image with 128 Colors'); imshow(IND,map);</pre> You may look at the value of map, where each row contains a color information: <pre>>> map(1,:) ans = 0.0157 0 0.0039</pre>	
Note: Indexed color saves a lot of memory, storage space, and transmission time: using truecolor, each pixel needs 24 bits, or 3 bytes. A typical 640x480 VGA resolution truecolor uncompressed image needs $640 \times 480 \times 3 = 921,600$ bytes (900 KiB). Limiting the image colors to 256, every pixel needs only 8 bits, or 1 byte each, so the example image now needs only $640 \times 480 \times 1 = 307,200$ bytes (300 KB), plus $256 \times 3 = 768$ additional bytes to store the palette map in itself (assuming RGB), approx. one third of the original size.	
<pre>img = imread('Penguins.jpg'); [IND1,map1] = rgb2ind(img,8); subplot(121); imshow(IND1,map1); title('Indexed image with 8 Colors');</pre> <pre>[IND2,map2] = rgb2ind(img,256); subplot(122); imshow(IND2,map2); title('Indexed image with 256 Colors');</pre>	Indexed image with 8 Colors  Indexed image with 256 Colors 

8.2.4. Gray to color image:

Color mapping:

MATLAB Code	Output
<pre>imshow(IND);</pre> (Improper Image without colormap)	

<pre>colormap(map)</pre> OR simply <pre>imshow(IND,map);</pre>	
---	--

Mapping grayscale image to jet and spring colormap

We have other built-in colormaps such as **jet** and **spring**. We may map grayscale image to these colormaps:

MATLAB Code	Output
<pre>img = imread('tower.png'); gray = rgb2gray(img); imshow(gray); colormap(jet)</pre>	
<pre>img = imread('tower.png'); gray = rgb2gray(img); imshow(gray); colormap(spring)</pre>	

Matlab's built-in colormaps

Matlab provides **built-in colormaps** as shown in the picture below:



Querying a pixel info

Because the image itself is an array, we can check the values it is storing. For example, if we want to know the values of the pixel at (350, 250):

MATLAB Code	Output
<code>img(350,250,:)</code>	<pre>ans(:,:,1) = 27 ans(:,:,2) = 19 ans(:,:,3) = 30</pre>
Also, there is a command imixelinfo. We type the command into the Command Window, and move the mouse on the image. Then, we'll see coordinates and RGB info (i.e. (455, 988) [255 241 12]) are being displayed while the mouse is on the picture. We can copy that info by right-mouse click and use it somewhere if necessary. <pre>imshow(img); imixelinfo;</pre> output will be different for indexed image: <pre>imshow(IND,map); imixelinfo;</pre>	

Lab Session 8.3: Filtering

When filtering an image, each pixel is affected by its neighbors, and the net effect of filtering is moving information around the image.

Image filtering can be grouped in two depending on the effects:

Low pass filters (Smoothing)	Low pass filtering (aka smoothing), is employed to remove high spatial frequency noise from a digital image. The low-pass filters usually employ moving window operator which affects one pixel of the image at a time, changing its value by some function of a local region (window) of pixels. The operator moves over the image to affect all the pixels in the image.
--	--

High pass filters (Edge Detection, Sharpening)	A high-pass filter can be used to make an image appear sharper. These filters emphasize fine details in the image - the opposite of the low-pass filter. High-pass filtering works in the same way as low-pass filtering; it just uses a different convolution kernel.
Mean Filter	Mean filtering is easy to implement. It is used as a method of smoothing images, reducing the amount of intensity variation between one pixel and the next resulting in reducing noise in images. The idea of mean filtering is simply to replace each pixel value in an image with the mean ('average') value of its neighbors, including itself. This has the effect of eliminating pixel values which are unrepresentative of their surroundings. Mean filtering is usually thought of as a convolution filter. Like other convolutions it is based around a kernel, which represents the shape and size of the neighborhood to be sampled when calculating the mean.

Important functions:

Function	What it does
<code>mf = ones(3,3)/9;</code>	<code>mf =</code> <pre> 0.1111 0.1111 0.1111 0.1111 0.1111 0.1111 0.1111 0.1111 0.1111 </pre> The mf is the mean filter:
<code>filter2()</code>	Y = filter2(h,X) filters the data in X with the two-dimensional FIR filter in the matrix h. It computes the result, Y, using two-dimensional correlation, and returns the central part of the correlation that is the same size as X.
<code>Y = filter2(h,X,shape)</code>	It returns the part of Y specified by the shape parameter. shape is a string with one of these values: 'full' : Returns the full two-dimensional correlation. In this case, Y is larger than X. 'same' : (default) Returns the central part of the correlation. In this case, Y is the same size as X. 'valid' : Returns only those parts of the correlation that are computed without zero-padded edges. In this case, Y is smaller than X.

Now we want to apply the kernel defined in the previous section using **filter2()**:

MATLAB Code	Output
<pre>img = imread('cameraman.tif'); imgd = im2double(img); f = ones(3,3)/9; img1 = filter2(f, imgd); subplot(121);imshow(img); title('original input') subplot(122);imshow(img1); title('blurred output')</pre>	original input  blurred output 
We can see the filtered image (right) has been blurred a little bit compared to the original input (left).	

As mentioned earlier, the low pass filter can be used denoising. Let's test it. First, to make the input a little bit dirty, we spray some pepper and salt on the image, and then apply the mean filter:

MATLAB Code	Output
<pre>img = imread('cameraman.tif'); imgd = im2double(img); imgd = imnoise(imgd, 'salt & pepper', 0.02); f = ones(3,3)/9; img1 = filter2(f, imgd); subplot(121);imshow(imgd); subplot(122);imshow(img1);</pre>	 

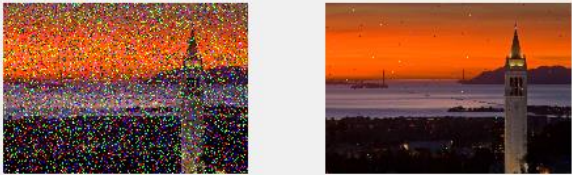
Using the Median filter - `medfilt2()`

MATLAB Code	Output
<pre>I = imread('cameraman.tif'); J = imnoise(I, 'salt & pepper', 0.02); K = medfilt2(J); subplot(121);imshow(J); subplot(122);imshow(K);</pre>	 
Much better. Unlike the previous filter which is just using mean value, this time we used median. Median filtering is a nonlinear operation often used in image processing to reduce "salt and pepper" noise. Also note that the medfilt2() is 2-D filter, so it only works for grayscale image.	

Removing noise in RGB image:

The filter we used to remove the "salt & pepper" type noise was **medfilt2()**. However, as the

"2" in the name indicates it's for 2-D array, it won't work for RGB image unless we decomposed each RGB channel and concatenate after the filtering each channel. That's exactly the following script does:

MATLAB Code	Output
<pre> I = imread('tower.png'); J = imnoise(I,'salt & pepper',0.2); % filter each channel separately r = medfilt2(J(:, :, 1), [3 3]); g = medfilt2(J(:, :, 2), [3 3]); b = medfilt2(J(:, :, 3), [3 3]); % reconstruct the image from r,g,b channels K = cat(3, r, g, b); figure subplot(121);imshow(J); subplot(122);imshow(K); </pre>	

Lab Session 8.4:Image Histogram and Histogram Equalization

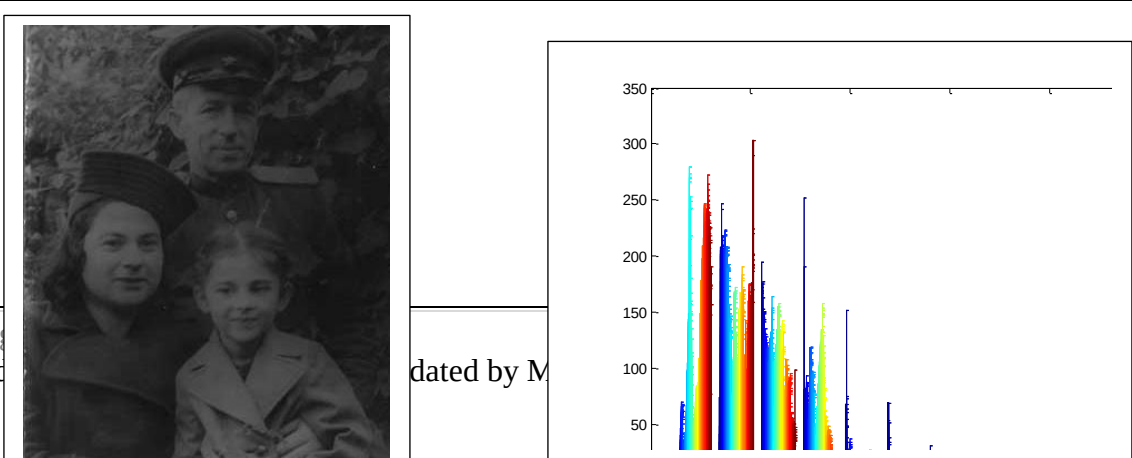
Image Histogram

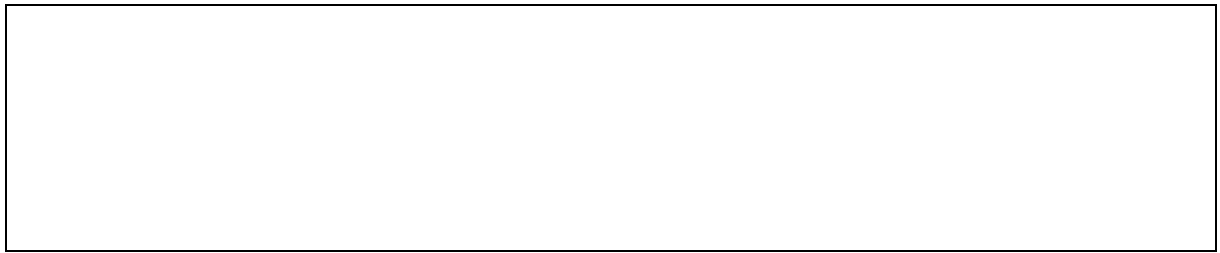
An image histogram is a type of histogram that acts as a graphical representation of the tonal distribution in a digital image. It plots the number of pixels for each tonal value. By looking at the histogram for a specific image a viewer will be able to judge the entire tonal distribution at a glance.

Used functions:

MATLAB functions	Use
imhist(img,n)	Display a histogram of image data. Displays a histogram with n bins for the intensity image above a grayscale colorbar of length n. If we omit the argument, imhist() uses a default value of n = 256 if I is a grayscale image, or n = 2 if I is a binary image.
histeq(img)	Enhance contrast using histogram equalization.

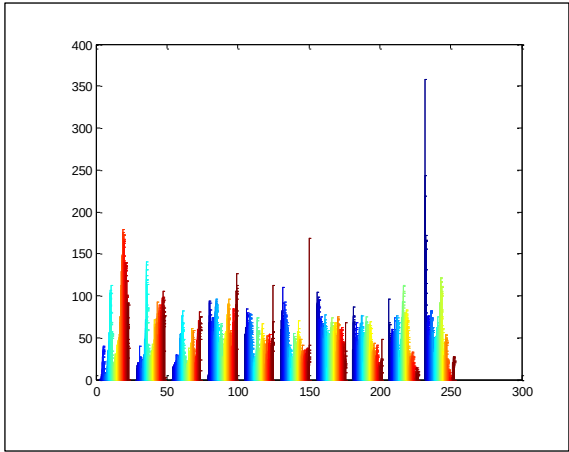
Histogram:

MATLAB Code	Output
<pre> I=imread ('family.jpg'); % read image file imshow (I) % show image I=double (I); % convert to 'double' type variable figure,hist (I) </pre>	The output is shown in the figure. 



Histogram Equalization

The Histogram Equalization algorithm enhances the contrast of images by transforming the values in an intensity image so that the histogram of the output image is approximately flat. See the picture below.

MATLAB Code	Output
<pre> I=imread ('family.jpg'); % read image file J=histeq (I); % histogram equalization imshow (J); % show image J=double (J); % convert to double type figure, hist (J); </pre>	The output is shown in the figure.
	

Comparison of images before and after equalization.

MATLAB Code	Output
-------------	--------


```
I = imread('family.jpg');  
J = histeq(I);  
subplot(121)  
imshow(I)  
subplot(122)  
imshow(J)
```



Home Work:

1. Apply the following noise models to any image (as you like) and use filtering operation for noise removal:
 - (i) Salt and pepper noise
 - (ii) Speckle noise
 - (iii) Gaussian noise
2. Write the following codes in a MATLAB script :

```
A=ones(16,16);  
A=A*255;  
B=zeros(16,16);  
C=[A B;B A];  
colormap(gray);image(C)
```

Explain and comment on the code.

Extend the above idea to draw a 8 by 8 chessboard in black and white.